NETRAX

Software guide: how it is built, how it works, how to fix and change it

Edge AI · Backend · Database · Dashboards · Firmware

Software report and maintenance guide

Team ZENTRIX · Team ID BH26PS06T044

BPUT Hackathon 2026 · Problem Statement 6

Prepared 12 November 2026

Contents

1 Software at a glance
2 Starting and stopping the system
3 How it works, step by step
4 Edge AI (folder `edge/`)
5 AI models and how they were trained
6 Backend server (folder `backend/`)
7 Database
8 Frontend: dashboards (folder `dashboard/`)
9 Firmware (ESP32 programs)
10 Technology: what is used for what
11 Folder map
12 Tools and tests
13 Troubleshooting: problem → what to do
14 Change guide: what to change, where
15 Safe working rules
16 Diagrams (draw.io files)

How to use this guide

Sections 1–3 explain the whole software in a few pages. Sections 4–12 describe every part and every file. Sections 13 and 14 are the practical part: when something goes wrong, or when you need to change a rule, they tell you which file to open and what to change. Every setting named here exists in the project exactly as written.

1 Software at a glance

NETRAX software has four parts that run on one laptop, plus small programs on two ESP32 boards. Everything talks over HTTP and WebSocket on the local network. No cloud is needed except for sending Telegram, e-mail and phone alerts.

Part	Folder	What it does	Built with
Edge AI	`edge/`	Reads every camera, runs the AI models on the GPU, tracks people, decides violations, sends alerts and live video to the backend.	Python, Ultralytics YOLOv8, PyTorch CUDA, OpenCV
Backend server	`backend/`	Receives alerts, video and sensor data; stores them; grades fire levels; sends notifications; serves the web pages and the API; login.	Python, FastAPI, Uvicorn, SQLite
Dashboards	`dashboard/`	Five web pages for the supervisor: control room, entrance, admin, hardware, safety intelligence, plus login.	HTML, CSS, plain JavaScript
Firmware	`firmware/`	ESP32 safety station (sensors) and ESP32 RFID reader.	Arduino C++
Settings	`config/`	All rules and thresholds in YAML files. Most changes need no code.	YAML
Models	`models/`	Trained AI model files.	YOLOv8 `.pt`, OpenCV `.onnx`
Data	`data/`	Database, snapshots, video clips, logs.	SQLite, JPEG, MP4

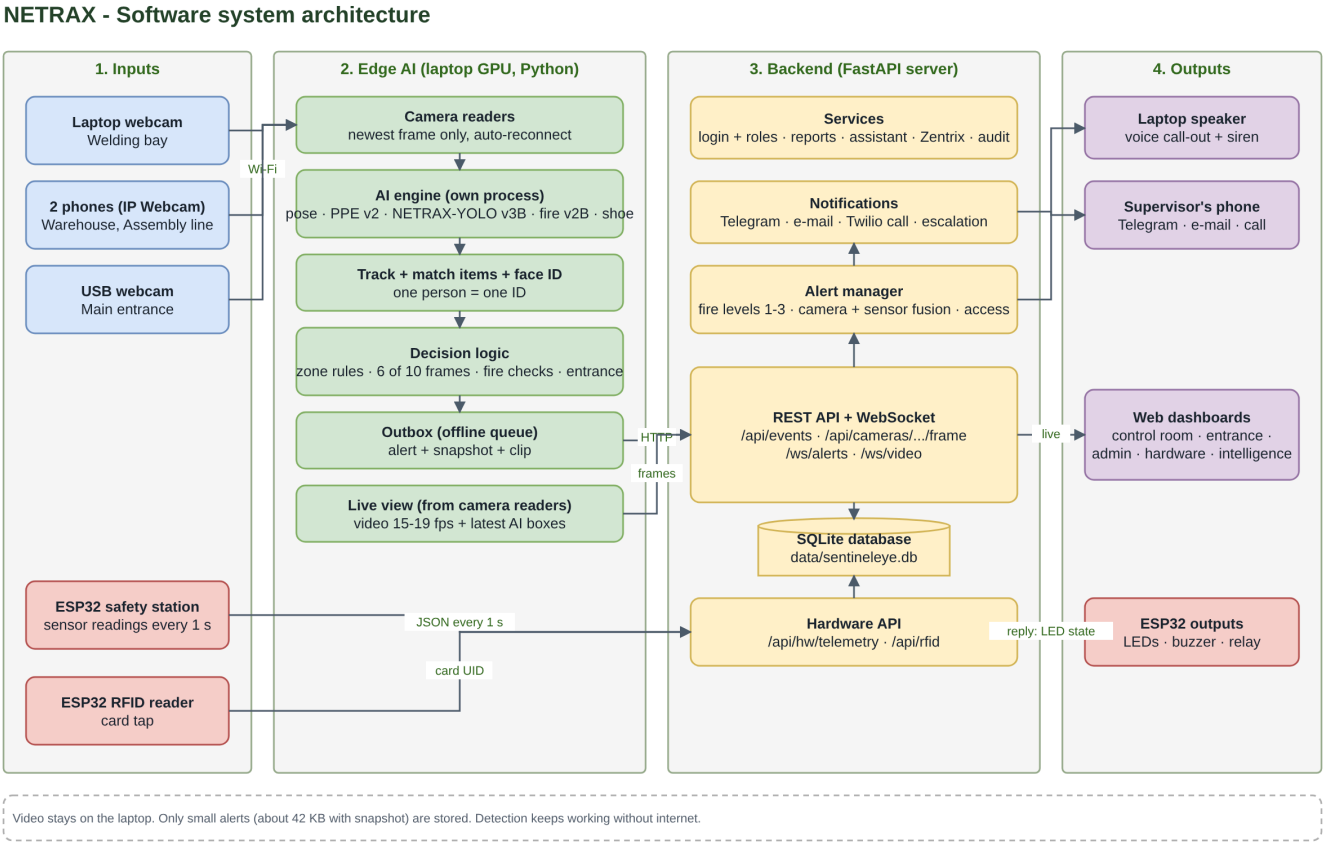


Figure 1.1 Software system architecture. The draw.io source is file `netrax_software_architecture.drawio` (section 16).

~20,000 lines of code (backend ~5,700, edge ~4,900, dashboards ~4,500, tools ~3,700)	**71** API routes plus 2 WebSocket channels	**14** database tables	**130** automated checks pass (final self-test)

2 Starting and stopping the system

2.1 One-time installation (new laptop)

1. Install Python 3.11 and the NVIDIA driver. Copy the project folder.
2. In the project folder: `python -m venv venv`, then `venv\Scripts\activate`.
3. Install PyTorch with CUDA (cu128 build from pytorch.org), then `pip install -r requirements.txt`.
4. Check that the model files are in `models/` (section 5).
5. Copy `config/secrets.example.yaml` to `config/secrets.local.yaml` and fill in Telegram, e-mail and Twilio (or run `python -m tools.notify_setup`).
6. First start creates the users `admin`, `supervisor`, `viewer` and writes their passwords once to `data/initial_passwords.txt`.
7. Run `python -m tools.final_selftest`. It must end with ALL FINAL TESTS PASSED.

2.2 Every day

Step	How
1. Prepare	Laptop on the charger (on battery the GPU slows to half speed). Phones: IP Webcam app open, screen on, same Wi-Fi. ESP32 boards powered.
2. Phone addresses changed?	`python -m tools.find_phones --write` finds the phones on the Wi-Fi and writes them into `config/cameras.yaml`.
3. Start	Double-click `run_all.bat`. It opens two windows (“NETRAX backend”, “NETRAX edge”) and the browser at `http://127.0.0.1:8000`.
4. Log in	Use the supervisor or admin account. From another device on the same Wi-Fi use `http://<laptop-IP>:8000`.
5. Wait	The edge needs about 20–40 s to load and warm up the models. Camera tiles turn LIVE.
6. Stop	Close the two black windows (or press Ctrl+C in each).

Manual start, for debugging, in two terminals with the venv active:

```
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
python -m edge.run_edge
```

3 How it works, step by step

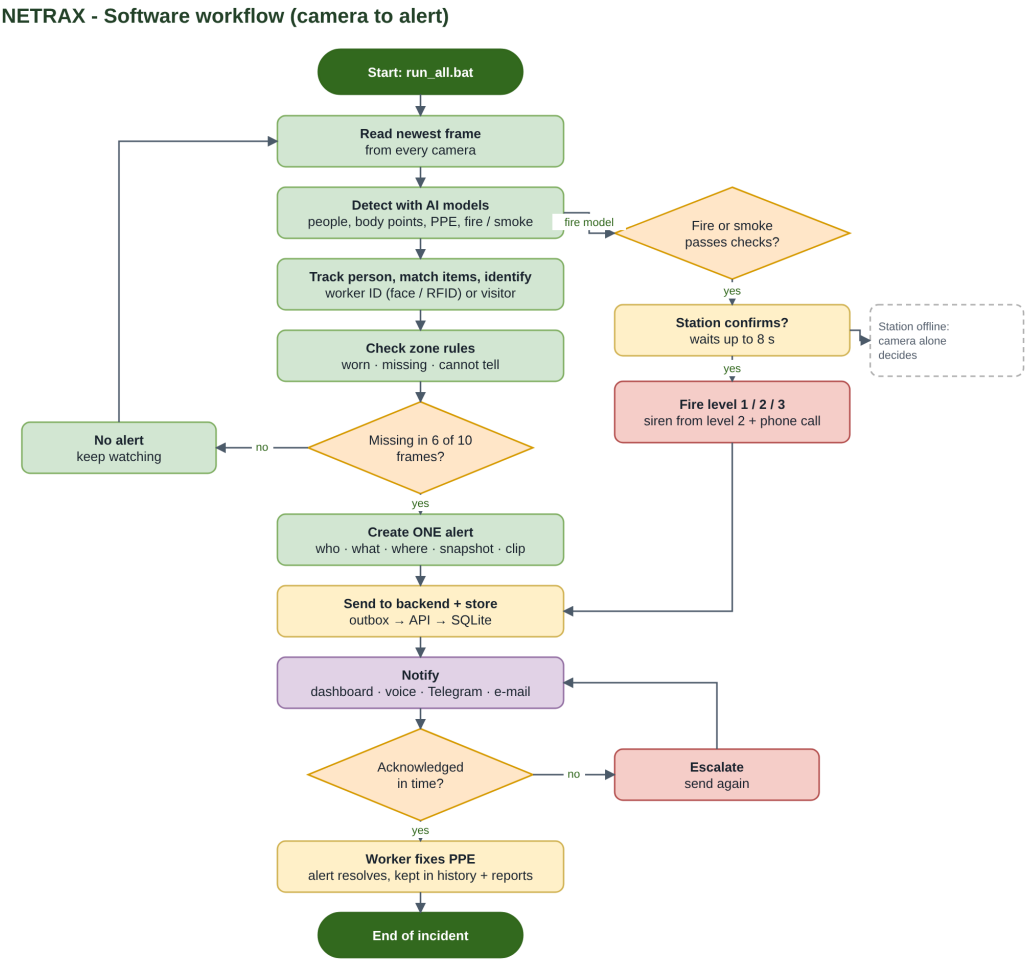


Figure 3.1 Software workflow from camera frame to alert. Draw.io source: `netrax_software_workflow.drawio`.

#	Step	What happens	File
1	Read frames	One thread per camera keeps only the newest frame; reconnects by itself.	`edge/stream.py`
2	Run the models	Frames of all cameras go to one AI process on the GPU, batched together.	`edge/engine_proc.py`, `detector.py`
3	Track	Each person keeps the same number between frames.	`edge/tracker.py`
4	Match items	Each helmet, vest, glove… is given to the right person using body points.	`edge/association.py`, `validate.py`
5	Identify	Face compared with registered workers in a background thread.	`edge/identity.py`
6	Decide	Zone rules + 6 of 10 frames + one decision per person.	`edge/state_machine.py`, `zones.py`
7	Fire check	Colour, flicker, growth, look-alike filters.	`edge/fire_verify.py`
8	Send	Alert + snapshot into an offline queue, then to the backend; video clip follows.	`edge/outbox.py`, `clips.py`
9	Live video	Newest frame + latest AI boxes sent at up to 16 fps, separate from the AI.	`edge/live.py`, `publisher.py`
10	Store + grade	Backend saves the event, merges camera and station evidence, sets fire level.	`backend/main.py`, `hardware.py`, `fire_levels.py`
11	Notify	Dashboard (WebSocket), voice, Telegram, e-mail, phone call; escalation.	`backend/notify.py`, `actuators.py`
12	Act	Supervisor acknowledges / resolves; everything goes to the audit log and reports.	`backend/main.py`, `reports.py`

4 Edge AI (folder `edge/`)

Started by `python -m edge.run_edge`. It runs as two processes: the main edge process (cameras, logic, sending) and the AI process (models on the GPU). They exchange frames through shared memory, so video and AI never block each other.

File	Job
`run_edge.py`	Entry point: loads settings, starts one reader and one worker per camera, the AI engine, live view, clips, outbox; prints status every 10 s; refuses to start twice.
`config.py`	Loads `config/settings.yaml`, `zones.yaml`, `cameras.yaml`.
`stream.py`	Camera reader thread: newest frame, reconnect with back-off, USB format (MJPG), black-picture detection.
`engine_proc.py` / `engine.py`	AI engine in its own process (default) / older in-process version (fallback). Batches frames of all cameras.
`detector.py`	Loads the models and runs them on a frame: pose → PPE main → PPE aux → fire → shoe classifier.
`worker.py`	Per-camera pipeline: detection → tracking → matching → zones → decisions → alerts; stage timing.
`tracker.py`	ByteTrack person tracking (simple IoU tracker as fallback).
`association.py`	Gives each item to its owner by body keypoints; worn / missing / cannot tell.
`validate.py`	Sanity checks: item size, vest must be hi-vis, helmet must sit on a head.
`state_machine.py`	Per-person decision: N-of-M frames, grace, one alert per look, update, resolve, re-arm.
`severity.py`	Severity (Low → Critical) from zone risk and items.
`zones.py`, `geometry.py`	Which zone a person stands in; box helpers.
`fire_verify.py`	Fire/smoke verification: colour, hot core, flicker, growth, look-alike rejection; writes `data/fire_debug.log`.
`identity.py`	Face detection (YuNet) + recognition (SFace), votes per track, visitor IDs; `data/identity_debug.log`.
`access.py`	Restricted zone: unauthorised registered workers.
`entrance.py`	Entrance gate decision: approved / denied / manual.
`live.py`, `annotate.py`	Smooth live view and drawing of boxes and labels.
`publisher.py`	Sends live frames and camera health to the backend.
`outbox.py`	Offline queue (`data/outbox.db`): alerts are never lost when the backend is down.
`clips.py`	4 s before + 4 s after each incident, faces blurred, MP4 upload.
`load.py`, `priority.py`	When the GPU is overloaded, slow low-risk cameras first; high-risk zones first.
`power.py`, `share.py`	Camera on/off from the dashboard; one device shared by two cameras.
`retention.py`	Deletes old snapshots (30 days by default).
`selftest.py`	Logic tests without cameras or models.

5 AI models and how they were trained

File in `models/`	Type	Detects	Setting that selects it
`yolov8n-pose.pt` (project root)	YOLOv8n-pose	Person + 17 body points	`models.person: pose`
`ppe_v2.pt`	YOLOv8s	Helmet, mask	`models.ppe`
`ppe_v3B.pt` (NETRAX-YOLO)	YOLOv8s + P2 head	Vest, gloves, goggles, head, face, hand, foot	`models.ppe_aux`, `aux_items`
`fire_smoke_v2B.pt`	YOLOv8n	Fire, smoke	`models.fire`
`shoe_cls.pt`	YOLOv8n-cls	Safety shoe / other footwear	`models.shoe_cls`
YuNet, SFace `.onnx`	OpenCV zoo	Face detection / recognition	fixed in `identity.py`

Older files (`ppe_v3A.pt`, `fire_smoke_v1.pt`, `fire_smoke_v2A.pt`) are kept for comparison; `v2A` is the automatic fallback if `v2B` is missing.

5.1 Training pipeline (what we did, in order)

#	Step	How	Tool / file
1	Record own videos	27 videos: PPE scenes, fire scenes, “no fire” look-alikes.	`datasets/raw_videos/`
2	Extract frames	1–2 frames per second, near-duplicates skipped.	`python -m tools.frames extract`
3	Pre-label	Our existing models draw first boxes.	`tools/frames.py`, `tools/collect.py`
4	Correct and review	Scene rules, colour checks, review on contact sheets.	`datasets/own/LABELS_README.md`
5	Split by scene	Whole videos go to train, val or test.	`datasets/splits.yaml`, `tools/build_dataset.py`
6	Stage A training	Public data (SH17, D-Fire, Roboflow) + dark/haze/blur copies on Colab T4.	`colab/*StageA.ipynb`
7	Stage B training	Stage A model + public + own scenes ×3. PPE 20 epochs, fire 10 epochs.	`colab/*StageB.ipynb`
8	Shoe classifier	Foot crops from our shoe videos, trained on the laptop.	`python -m tools.shoes train`
9	Evaluate	Score on our unseen test scenes; speed on the laptop.	`tools/eval_hardtest.py`, `tools/benchmark.py`
10	Deploy	Copy the `best.pt` into `models/`, point the setting to it, run the self-tests.	`config/settings.yaml`

Results (own unseen test scenes): PPE v3B mAP50 0.959, fire v2B 0.870. Details: `docs/model_card.md`, `docs/eval/`. Our model layout with the extra P2 head is `colab/sentineleye-yolov8s-p2.yaml`. All models run in FP32 (the GTX 1650 has problems with FP16).

6 Backend server (folder `backend/`)

Started by `python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000`. It serves the web pages and the API on port 8000.

File	Job
`main.py`	The web app: all routes, event ingest, acknowledge / resolve / escalate, camera frames, WebSockets, health, Zentrix execution, background loops.
`db.py`	Creates and opens the SQLite database (`data/sentineleye.db`).
`auth.py`	Login, sessions, roles (viewer / supervisor / admin), which path needs which role, audit log.
`notify.py`	Telegram, e-mail, Twilio call/SMS, CallMeBot, Android call; rules, retries, cooldown, escalation; delivery status.
`actuators.py`	Windows voice call-outs and siren on the laptop speaker.
`fire_levels.py`	Fire level 1 / 2 / 3 from size, growth, duration, sensors, sources.
`hardware.py`	Station telemetry evaluation, learned baselines, camera + sensor fusion, RFID attendance logic.
`hw_api.py`	Hardware routes: `/api/hw/telemetry` , `/api/hw/rfid` , `/api/rfid` , attendance.
`reports.py`	Analytics, CSV and PDF reports, custom reports.
`assistant.py`	History questions → database query → answer only from records.
`agent.py`	Zentrix command parser: commands → planned actions (needs confirmation for risky ones).
`location.py`	Location per incident (hardware GPS, browser, or site setting).
`acceptance.py`	Acceptance test checklist (tests 01–14).
`perf.py` , `timeutil.py`	Latency statistics; one time zone (IST) everywhere.

6.1 Main API groups

Route(s)	Used by / for
`POST /api/events`, `/api/events/{id}/clip`	Edge → new / updated alert, video clip
`POST /api/cameras/{id}/frame`, `/api/edge/health`	Edge → live picture, camera health
`GET /api/events`, `/api/events/search`	Dashboards → incident lists and search
`POST /api/events/{id}/ack | resolve | escalate | dismiss`	Supervisor actions
`POST /api/hw/telemetry`	ESP32 safety station (no login)
`POST/GET /api/rfid`, `/api/hw/rfid`	ESP32 RFID reader (no login)
`/api/workers…`	Register workers, photos, link badges (admin)
`/api/reports/custom`, `/api/reports/export`	PDF / CSV reports
`POST /api/zentrix`, `/api/assistant`	Zentrix commands and history questions
`POST /api/cameras/{id}/power`	Camera on/off
`GET /api/health`, `/api/perf`, `/api/notify/status`	System health, latency, channel status
`POST /api/login`, `/api/logout`, `GET /api/me`	Login
`WS /ws/alerts`, `WS /ws/video`	Live alerts and live video to the browser

7 Database

One SQLite file: `data/sentineleye.db` (created by `backend/db.py`). It can be opened with “DB Browser for SQLite”. Close NETRAX before editing it, and make a copy first.

Table	What it stores	Rows (11 Nov)
`events`	Every incident: type, missing items, severity, camera, zone, worker, status, times, snapshot, clip, fire level, location	1,178
`event_actions`	Every acknowledge / resolve / escalate / dismiss, by whom and when	5,288
`notifications`	Each Telegram / e-mail / call: recipient, status, attempts, provider reply	1,915
`announcements`	Voice call-outs spoken on the speaker	2,542
`workers`	Registered workers: ID, name, face embeddings, photos, RFID card	6
`attendance`	RFID entries and exits	31
`entrance_checks`	Each entrance decision with items and gate state	195
`hw_readings`	Safety station readings (one row per 10 s)	3,349
`audit_log`	Logins and every action by every user	319
`assistant_log`	Assistant questions and answers	34
`locations`	Location readings	17
`acceptance`	Acceptance test results	0
`rfid_scans`, `sensor_readings`	Older simple-format tables (kept for compatibility)	0

Other data on disk: `data/backend_snapshots/` (alert pictures, faces blurred), `data/backend_clips/` (MP4 clips), `data/workers/` (registration photos), `data/outbox.db` (edge queue), logs (`fire_debug.log`, `identity_debug.log`), `data/camera_power.json` (camera on/off).

8 Frontend: dashboards (folder `dashboard/`)

Plain HTML, CSS and JavaScript. No build step, no internet. The backend serves each page; shared files are under `/static/`.

File	Address	What it shows
`index.html`	`/`	Control room: KPIs, camera wall, entrance, zones and people, station, health, fire box, PPE box
`entrance.html`	`/entrance`	Gate camera, today's counts, decisions, manual allow / refuse
`admin.html`	`/admin`	Incidents, workers, reports, alert channels, cameras and zones, users and audit, health, acceptance
`hardware.html`	`/hardware`	Station state, sensor graphs, attendance, fire cross-check log
`intelligence.html`	`/intelligence`	Overview, sensors, AI monitoring, workers, alerts centre, forecast, factory map, device health, reports
`login.html`	`/login`	Login page
`video.js`	all pages	Live video over `/ws/video` into every `<img data-live=…>`
`alerts.js`	all pages	Alert side panel (fire, PPE, entrance groups)
`zentrix.js`	all pages	Zentrix panel: typed and spoken commands
`nx.css`, `theme.css`, `theme.js`	all pages	Look, colours, day / night switch
`roi.html`	`/static/roi.html`	Cost and payback calculator (example inputs)

Important after changing any dashboard file

Browsers keep old copies. After editing a `.js` or `.css` file, raise the version in the links (for example `?v=5nov2` → `?v=12nov1`) in every page that loads it, then press Ctrl+F5 in the browser.

9 Firmware (ESP32 programs)

Folder	Program
`firmware/netrax_station_team/`	**In use.** Team safety station: reads DHT22, flame, MQ-2, MQ-135, FSR, dust, MPU6050 every second; POSTs JSON to `/api/hw/telemetry` as `esp32-ppe-01`; sets LEDs, buzzer and relay from the reply.
`firmware/rfid_direct/`	**In use.** RFID reader: sends every card UID to `/api/rfid`; beeps and shows the server's answer.
`firmware/netrax_esp32/`	Our first full station sketch (reference).
`firmware/rfid_reader/`	Older Google-Sheets RFID sketch (reference).

Each folder has `secrets.h` (Wi-Fi name, password, laptop address; private) and `secrets.example.h` (template). Upload with Arduino IDE, board "ESP32 Dev Module".

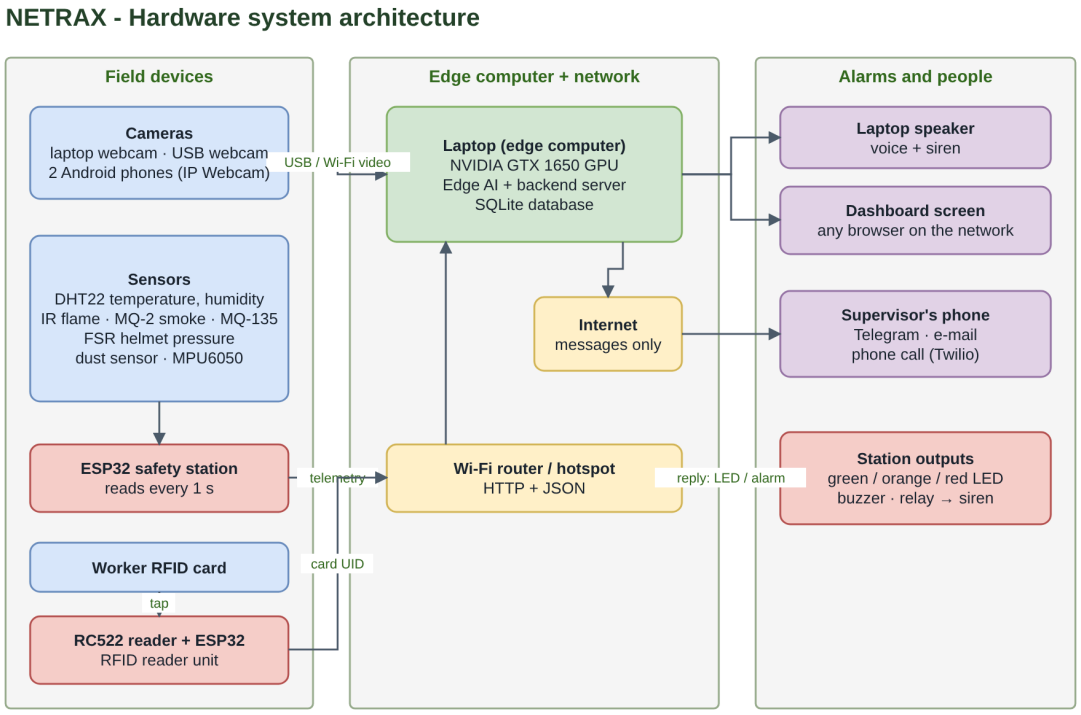


Figure 9.1 Hardware system architecture (`netrax_hardware_architecture.drawio`).

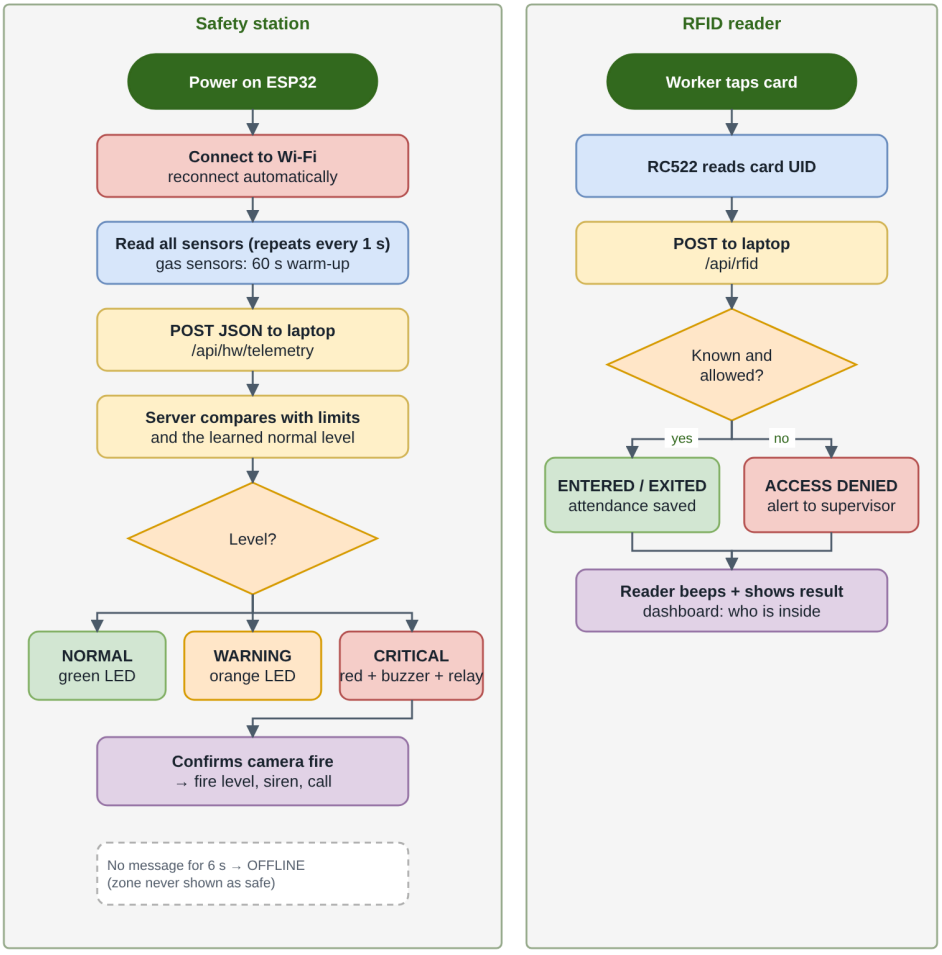


Figure 9.2 Hardware workflow (`netrax_hardware_workflow.drawio`).

10 Technology: what is used for what

Technology	Used for	Where
Python 3.11	Edge AI, backend, tools	`edge/`, `backend/`, `tools/`
Ultralytics YOLOv8 + PyTorch CUDA	Running and training the detection models	`edge/detector.py`, `colab/`
OpenCV	Camera reading, drawing, colour checks, YuNet + SFace faces, video	`edge/`
Supervision (ByteTrack)	Person tracking	`edge/tracker.py`
NumPy, shared memory, multiprocessing	Fast frame passing to the AI process	`edge/engine_proc.py`
FastAPI + Uvicorn	Web server, API, WebSockets	`backend/main.py`
SQLite	Database (one file)	`backend/db.py`
PyYAML	All settings files	`config/`
Requests	Edge → backend, Telegram, Twilio, CallMeBot	`edge/`, `backend/notify.py`
smtplib (Gmail SMTP)	E-mail alerts	`backend/notify.py`
Windows SAPI (pywin32), winsound	Voice call-outs, siren	`backend/actuators.py`
ReportLab	PDF reports	`backend/reports.py`
PyAV (H.264)	Incident video clips	`edge/clips.py`
HTML, CSS, JavaScript, SVG	Dashboards and charts	`dashboard/`
Web Speech API (browser)	Speaking to Zentrix (needs Edge/Chrome + internet)	`dashboard/zentrix.js`
Arduino C++ on ESP32	Station and RFID firmware	`firmware/`
Google Colab, Roboflow	Training, public datasets	`colab/`
IP Webcam (Android app)	Phones as cameras	`config/cameras.yaml`

11 Folder map

Folder / file	Contents
`run_all.bat`	Starts backend + edge + browser
`SETUP_PHONE_CALL.bat`	Optional set-up for an Android SIM call (not used with Twilio)
`requirements.txt`	Python packages
`CLAUDE.md`, `README_EDGE.md`	Developer notes: full change history
`edge/`	Edge AI (section 4)
`backend/`	Server (section 6)
`dashboard/`	Web pages (section 8)
`config/`	Settings (section 14)
`models/`	Model files
`firmware/`	ESP32 programs
`tools/`	Tests, set-up and measuring tools (section 12)
`datasets/`	Own videos, frames, labels, splits
`colab/`	Training notebooks and the NETRAX-YOLO model layout
`docs/`	Model card, evaluation results, guides, reports, failure gallery
`data/`	Database, snapshots, clips, logs, worker photos (private)
`runs/`	Training / validation outputs on the laptop
`venv/`	Python environment (do not edit)
`_to_delete/`	Backups of replaced files and test leftovers; safe to delete after the final

12 Tools and tests

Run each from the project folder with the venv active: `python -m tools.<name>`.

Command	Use it when
`tools.final_selftest`	After any change. Must end ALL FINAL TESTS PASSED (stop NETRAX first; two timing checks fail if the GPU is busy).
`edge.selftest`, `tools.hw_selftest`, `tools.access_selftest`	After changes to edge logic, hardware logic or access rules.
`tools.find_phones --write`	Phone camera addresses changed (new Wi-Fi).
`tools.phone_tune --apply`	Phone video laggy: sets 960×540, quality 55.
`tools.camdiag`	Video freezes: tests cameras without AI (camera / Wi-Fi problem or processing problem).
`tools.notify_setup`	Set up or test Telegram, e-mail, Twilio.
`tools.users list | set <name> <password> <role>`	Show users, change a password, add a user.
`tools.hw_sim`	No ESP32 at hand: simulates the station and RFID reader.
`tools.latency_report --label …`	Measure delays on the running system.
`tools.benchmark`	Measure model speed (NETRAX stopped).
`tools.eval_hardtest`, `tools.check_all`	Score models on our own test scenes; all checks in one run.
`tools.gallery`	Rebuild the failure-case gallery.

13 Troubleshooting: problem → what to do

First look at the two black windows. The edge window prints a status line every 10 s (camera fps, AI checks per second, backend up/down) and `[perf]` timing lines. Most answers are there.

13.1 Starting and login

Problem	Likely cause	What to do (where)
Browser: “can’t reach 127.0.0.1:8000”	Backend not running or crashed	Read the backend window for the error. Start it again. Port busy → close the old window or restart the laptop.
Edge window closes or says another copy is running	NETRAX started twice	Close all NETRAX windows, start `run_all.bat` once.
“No module named …”	venv not active or package missing	`venv\Scripts\activate`, then `pip install -r requirements.txt`.
Forgot a password	—	`python -m tools.users set admin NewPass admin`. Passwords are stored hashed in `config/users.local.yaml`.
Page shows old design after a change	Browser cache	Raise `?v=` in the page links (section 8), Ctrl+F5.
Phone / another PC cannot open the dashboard	Different network or firewall	Same Wi-Fi; use `http://<laptop-IP>:8000`; allow Python in Windows Firewall (private and public).

13.2 Cameras and video

Problem	Likely cause	What to do (where)
Phone camera OFFLINE	Phone IP changed, app closed, screen off	Open IP Webcam, “Start server”, keep screen on; `python -m tools.find_phones --write`; restart the edge.
Laptop and USB camera swapped	Windows changed device order	`config/cameras.yaml`: swap `source: 0` and `source: 1`.
USB camera black or 1 fps	Loose cable, power saving, lens covered	Re-plug; the edge reopens a black camera by itself after 8 s. USB selective suspend is already off in the power plan.
Video hangs / low fps	On battery, phone resolution too high, weak Wi-Fi	Charger in; `tools.phone_tune --apply`; run `tools.camdiag` to see if the camera itself freezes.
AI slow (“AI 1/s” on the tile)	GPU busy, battery, too many cameras	Charger; turn off a camera from the dashboard; `processing.max_batch` / `target_fps` in `settings.yaml`.
Phone picture sideways	Phone held upright	`cameras.yaml`: `rotate: 90` (or 0 / 180 / 270).
Camera shows OFF	Switched off from the dashboard	Press “Turn on” on the tile (saved in `data/camera_power.json`).

13.3 Detection

Problem	Likely cause	What to do (where)
Too many PPE false alarms	Low confidence, small / far people	`settings.yaml`: raise `thresholds.ppe_conf` for that item, or `state_machine.by_risk.*.confirm_n`; check snapshots in `data/backend_snapshots/`.
Alert comes too late	Confirmation waits	Lower `first_look_s`, `grace_s`, `settle_s` in `state_machine.by_risk` (more false alarms possible).
Item always “cannot tell”	Body part not visible (feet out of view)	Move the camera to see the whole person; that is correct behaviour, not an error.
Wrong worker name	Few or poor registration photos	Admin → Workers: register 3–5 photos in different light; see `data/identity_debug.log`; `identity.match_threshold`.
Fire false alarm (lamp, shirt, steam)	Look-alike passed the filter	Read `data/fire_debug.log` (why it was confirmed); adjust `fire.*` in `settings.yaml`; a station in that zone prevents it.
Real fire not alarmed	Station online but did not confirm	Event shows “suppressed”. Check the station sensors; `hardware.yaml` `fusion.verify_window_s`.
Entrance always “manual check”	Person too far, two people, no face	Stand closer, one at a time; `settings.yaml` `entrance.min_height_frac`.

13.4 Hardware, alerts, data

Problem	Likely cause	What to do (where)
Station OFFLINE	Wrong Wi-Fi or laptop IP in firmware	Serial monitor (115200); fix `secrets.h` (Wi-Fi, laptop IP), re-upload; `DEVICE_ID` must match `hardware.yaml devices`.
Station always CRITICAL / buzzer on	Thresholds not calibrated for your sensors	`config/hardware.yaml thresholds` (look at real values on /hardware first).
RFID tap does nothing	Reader URL / Wi-Fi wrong, card not linked	Fix `secrets.h` of `rfid_direct`; Admin → Workers → link card.
Telegram / e-mail not arriving	Missing or wrong credentials	`python -m tools.notify_setup`; Admin → Alerts & channels → test; the delivery log shows the provider’s error.
Phone call not ringing	Twilio trial: number not verified	Verify the number in Twilio console or upgrade; `secrets.local.yaml twilio.to_numbers`.
No voice on the speaker	Muted or turned off	`settings.yaml response.speaker.enabled`; Windows volume; dashboard bell button.
Alerts not on dashboard but edge works	Backend down → alerts queued	Start the backend; the outbox sends queued alerts automatically.
Disk full	Snapshots and clips	`privacy.snapshot_retention_days` in `settings.yaml`; delete old files in `data/`.

14 Change guide: what to change, where

Most changes are in `config/` and need only a restart of the edge or backend. Make a copy of the file first.

I want to…	File	What to change
Add or move a camera	`config/cameras.yaml`	New block `cam_05` with `name`, `source` (index or `http://…/video`), `default_zone`, `rotate`. `enabled: false` hides one.
Change required PPE in a zone	`config/zones.yaml`	`required_ppe: [helmet, vest, gloves, goggles, boots, mask]` for that zone.
Add a zone	`config/zones.yaml` (+ `site.yaml`)	New block with `name`, `required_ppe`, `risk: high | medium | low`; point a camera's `default_zone` to it.
Change who may enter the welding bay	`config/zones.yaml`	`allowed_workers: [W-001, W-003]`. Remove the line to open the zone.
Make alerts faster or stricter	`config/settings.yaml`	`state_machine.by_risk.high`: `confirm_n`, `grace_s`, `first_look_s`, `settle_s`.
Detection confidence	`config/settings.yaml`	`thresholds.ppe_conf` per item, `thresholds.fire_conf` (Fire 0.45, Smoke 0.35).
Use a new model	`models/` + `settings.yaml`	Copy the `.pt` file; set `models.ppe_aux` / `models.fire` / `models.ppe`; run the self-tests and `tools.eval_hardtest`.
Which channels get which alert	`config/notify.yaml`	`rules:` (e.g. `ppe_missing_High: [telegram, email]`); `enabled_channels`.
Who receives messages / calls	`config/secrets.local.yaml`	`telegram.chat_id`, `email.to`, `twilio.to_numbers`.
Phone call provider	`config/notify.yaml`	`phone.provider: twilio` (or `android,callmebot`).
Link in messages for phones	`config/notify.yaml`	`dashboard_url: http://<laptop-IP>:8000`.
Fire level rules / siren	`config/settings.yaml`	`fire_levels:` (e.g. `siren_from_level: 2`).
Sensor limits	`config/hardware.yaml`	`thresholds: temp_critical_c`, `mq2_abs`, `mq2_rise`, `pm25_warn` …
Add a station or reader	`config/hardware.yaml` + firmware	`devices: esp32-ppe-02: {name, zone_id}` or `attendance.readers`; same ID in the sketch.
New Wi-Fi / laptop IP for ESP32	`firmware/…/secrets.h`	Wi-Fi name, password, laptop IP; re-upload.
Register a worker / card	Dashboard	Admin → Workers: photos (consent) and RFID card.
Users and passwords	`tools.users`	`python -m tools.users set <name> <password> <role>`.
Live video rate / size	`config/settings.yaml`	`processing.live_fps`, `live_width`.

Voice and siren on/off	`config/settings.yaml`	`response.speaker`, `response.alarm`.
Snapshot retention / face blur	`config/settings.yaml`	`privacy.snapshot_retention_days`, `blur_faces_in_snapshots`.
Site name and location	`config/site.yaml`	`site:` `name`, `address`, `lat`, `lon`; `zones:` descriptions.
Dashboard text or layout	`dashboard/*.html`	Edit the page; raise `?v=`; Ctrl+F5.
A new Zentrix command	`backend/agent.py` (+ `main.py _zx_execute`)	Add words to the plan rules; add a test in `tools/final_selftest.py` [ZX].
A new API route	`backend/main.py` (+ `auth.py`)	Add the route; set which role may use it in `auth.py`.

15 Safe working rules

- **Back up before you change.** Copy the file (or the database) into `_to_delete/` with a date.
- **Test after you change.** Run `python -m tools.final_selftest` with NETRAX stopped. Every bug fix gets a test.
- **Never put secrets on GitHub.** `config/secrets.local.yaml`, `config/users.local.yaml`, `config/hardware.local.yaml`, `firmware/*/secrets.h` and `data/` are in `.gitignore`. Before any push, revoke the old Gmail app password that was once in an earlier commit.
- **Do not rename** the folder, `data/sentineleye.db` or the `SENTINELEYE_*` environment variables: code depends on them.
- **Do not change the alert format** sent by the edge (fields may be added, never renamed or removed).
- **Only “missing” may raise a PPE alert;** “cannot tell” never does.
- **Run models in FP32** on the GTX 1650; do not retrain without a plan and a test set.
- **Only claim measured numbers.** Label prototype values (sensor thresholds, cost inputs) as such.

16 Diagrams (draw.io files)

Four diagram files come with this report. They are plain draw.io XML: open app.diagrams.net (or draw.io desktop / the VS Code “Draw.io Integration” extension) → File → Open from → Device → choose the `.drawio` file. You can also paste the XML through Extras → Edit Diagram. Every box can be moved and edited.

File	Shows
`netrax_software_architecture.drawio`	Inputs → edge AI → backend → outputs (Figure 1.1)
`netrax_software_workflow.drawio`	Camera frame → decision → alert → notify → escalate / resolve, with the fire branch (Figure 3.1)
`netrax_hardware_architecture.drawio`	Sensors, ESP32 boards, cameras, Wi-Fi, laptop, alarms, supervisor (Figure 9.1)
`netrax_hardware_workflow.drawio`	Safety station loop and RFID tap flow (Figure 9.2)

Colours used in all four: blue = inputs, green = edge AI, yellow = backend / network, purple = outputs, red = hardware and alarms, orange diamond = decision.